École Nationale des Sciences Appliquées

ENSA

Compte Rendu TP3
Architecture Distribuée

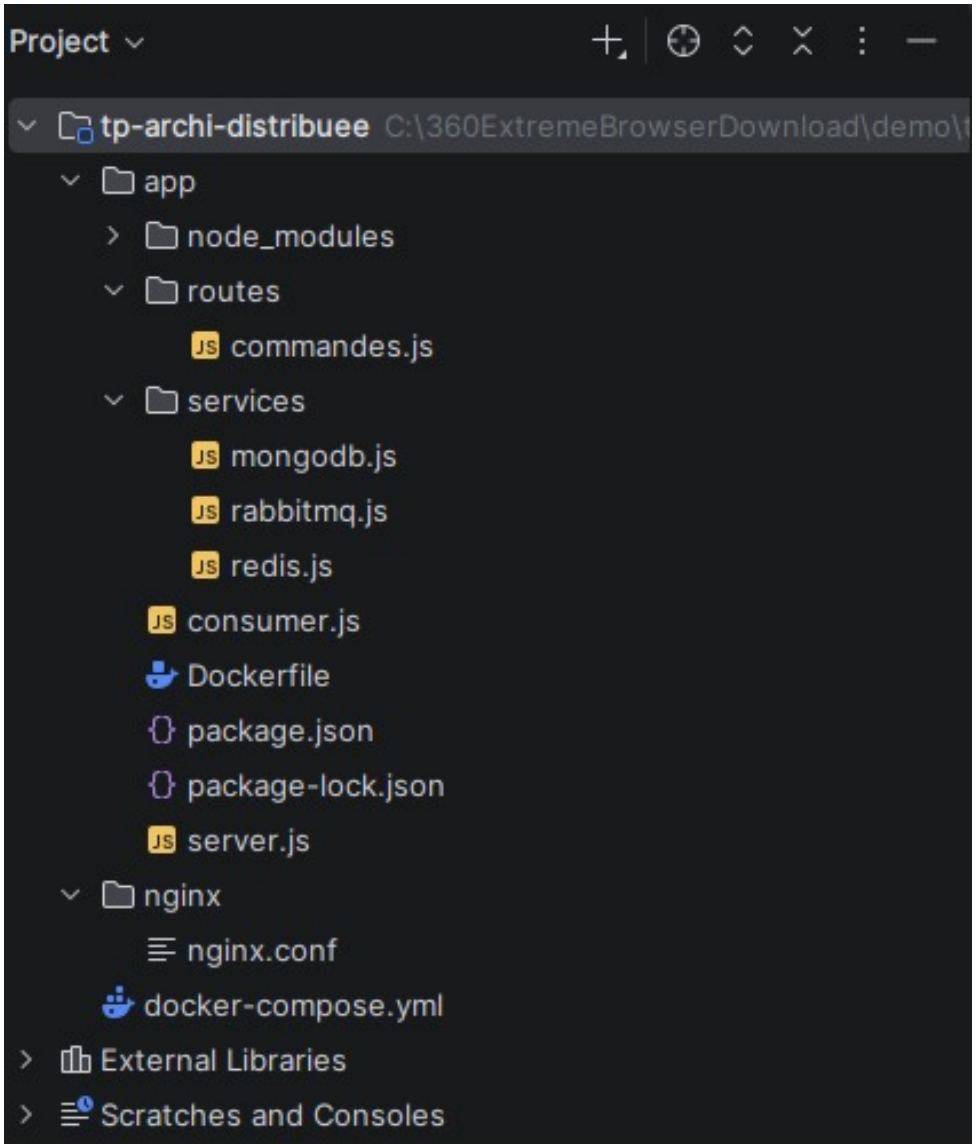
Réalisé par :
Kaya Hind

Encadré par :
B. El Baghzaoui

1 Introduction

Ce travail pratique consiste à concevoir et déployer une architecture distribuée basée sur plusieurs services interconnectés. L'objectif principal est de comprendre comment différentes technologies peuvent collaborer pour assurer la gestion, le stockage, la mise en cache et la communication des données dans un système moderne.

2 Structure du projet



La structure du projet est organisée en plusieurs dossiers afin de séparer les différentes responsabilités. Le dossier `app` contient la logique applicative, répartie entre les routes et les services, tandis que le dossier `nginx` regroupe la configuration du serveur de répartition de charge. Cette organisation améliore la lisibilité et facilite la maintenance du projet.

3 Configuration Docker

docker-compose.yml

```yaml
version: '3.8'

services:

  # MongoDB
  mongodb:
    image: mongo:6
    container_name: mongodb
    ports:
      - '27017:27017'
    volumes:
      - mongo_data:/data/db

  # Redis
  redis:
    image: redis:7-alpine
    container_name: redis
    ports:
      - '6379:6379'

  # RabbitMQ
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - '5672:5672'
      - '15672:15672'
```

Le fichier `docker-compose.yml` permet de définir l'ensemble des services nécessaires au fonctionnement du système. Il orchestre le déploiement de MongoDB pour le stockage, Redis pour le cache, RabbitMQ pour la messagerie, ainsi que deux instances de l'application Node.js. Cette configuration permet de lancer toute l'architecture avec une seule commande.

4 Configuration Nginx

nginx.conf

```
events {
  worker_connections 1024;
}

http {
  upstream backend {
    server app1:3001;
    server app2:3002;
  }

  server {
    listen 80;

    location / {
      proxy_pass http://backend;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
    }
  }
}
```

Nginx est configuré comme un répartiteur de charge (load balancer). Il distribue les requêtes entrantes entre les différentes instances Node.js selon une stratégie de type round-robin. Cela permet d'équilibrer la charge du système et d'améliorer la disponibilité de l'application.

5 Connexion à MongoDB

mongodb.js

```
const mongoose = require('mongoose');

const CommandeSchema = new mongoose.Schema({
  produit:  { type: String, required: true },
  quantite: { type: Number, required: true },
  client:   { type: String, required: true },
  statut:   { type: String, default: 'en_attente' },
  date:     { type: Date,   default: Date.now }
});

const Commande = mongoose.model('Commande', CommandeSchema);

async function connectMongo() {
  await mongoose.connect('mongodb://mongodb:27017/boutique');
  console.log('✅ MongoDB connecté');
}

module.exports = { connectMongo, Commande };
```

Ce fichier met en place la connexion à la base de données MongoDB et définit le modèle de données utilisé pour les commandes. MongoDB joue un rôle essentiel en assurant le stockage persistant des informations, ce qui garantit la conservation des données même après redémarrage du système.

6 Connexion à Redis

redis.js

```
const Redis = require('ioredis');
const redis = new Redis({ host: 'redis', port: 6379 });
redis.on('connect', () => console.log('✅ Redis connecté'));
module.exports = redis;
```

Redis est utilisé comme système de cache en mémoire. Lorsqu'une requête de lecture est effectuée, les données sont d'abord recherchées dans Redis. Si elles sont présentes, cela permet de répondre rapidement sans solliciter la base de données. Ce mécanisme améliore significativement les performances du système.

7 Connexion à RabbitMQ

```js
const amqp = require('amqplib');

let channel;
const QUEUE = 'commandes';

async function connectRabbitMQ() {
  const conn = await amqp.connect('amqp://admin:admin@rabbitmq');
  channel = await conn.createChannel();
  await channel.assertQueue(QUEUE, { durable: true });
  console.log('✅ RabbitMQ connecté');
}

function publierCommande(commande) {
  const msg = JSON.stringify(commande);
  channel.sendToQueue(QUEUE, Buffer.from(msg), { persistent: true });
  console.log('📤 Message publié dans RabbitMQ');
}

module.exports = { connectRabbitMQ, publierCommande };
```

RabbitMQ est intégré afin de gérer la communication asynchrone entre les différents composants. Lorsqu'une commande est créée, un message est envoyé dans une file d'attente, ce qui permet de déclencher des traitements indépendants du flux principal.

8 Consumer RabbitMQ

```js
const amqp = require('amqplib');

async function demarrerConsommateur() {
  const conn = await amqp.connect('amqp://admin:admin@rabbitmq');
  const channel = await conn.createChannel();
  const QUEUE = 'commandes';
  await channel.assertQueue(QUEUE, { durable: true });

  console.log('👂 Consommateur en attente de messages...');
  channel.consume(QUEUE, (msg) => {
    if (msg) {
      const commande = JSON.parse(msg.content.toString());
      console.log('🔔 NOUVELLE COMMANDE REÇUE !');
      console.log('   Client  :', commande.client);
      console.log('   Produit :', commande.produit);
      console.log('   ID      :', commande.id);
      console.log('📧 Email de confirmation envoyé (simulé)\n');
      channel.ack(msg);
    }
  });
}

setTimeout(demarrerConsommateur, 5000);
```

Le consommateur (consumer) est responsable de la récupération des messages envoyés via RabbitMQ. Il traite ces messages en simulant des actions telles que l'envoi d'un email de confirmation. Ce mécanisme illustre le fonctionnement des systèmes distribués basés sur des événements.

9 Dockerfile

```
Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "server.js"]
```

Le Dockerfile permet de créer une image contenant l'application Node.js ainsi que toutes ses dépendances. Il définit les étapes nécessaires pour construire et exécuter le service dans un conteneur, garantissant ainsi un environnement cohérent et reproductible.

10 Fichier package.json

```
package.json
{
  "name": "app",
  "version": "1.0.0",
  "description": "",
  "main": "consumer.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "amqplib": "^0.10.9",
    "express": "^5.2.1",
    "ioredis": "^5.10.1",
    "mongoose": "^9.3.1"
  }
}
```

Le fichier `package.json` contient l'ensemble des dépendances nécessaires au fonctionnement de l'application. Il permet également de gérer les scripts et la configuration du projet Node.js.

11 Serveur Node.js

server.js

```javascript
const express = require('express');
const { connectMongo } = require('./services/mongodb');
const { connectRabbitMQ } = require('./services/rabbitmq');
const commandesRoutes = require('./routes/commandes');

const app = express();
const PORT = process.env.PORT || 3001;
const INSTANCE = process.env.INSTANCE || 'Instance-?';

app.use(express.json());
app.use('/commandes', commandesRoutes);

app.get('/health', (req, res) => res.json({ status: 'OK', instance: INSTANCE }));

async function start() {
  await connectMongo();
  await connectRabbitMQ();
  app.listen(PORT, () => console.log(`🚀 ${INSTANCE} lancée sur le port ${PORT}`));
}

start();
```

Le serveur Node.js constitue le cœur de l'application. Il gère les requêtes HTTP, interagit avec MongoDB pour le stockage, utilise Redis pour le cache et communique avec RabbitMQ pour l'envoi de messages. Il joue ainsi un rôle central dans la coordination de tous les services.

12 Conclusion

Ce TP a permis de mettre en œuvre une architecture distribuée complète en intégrant plusieurs technologies complémentaires. L'utilisation combinée de Docker, MongoDB, Redis, RabbitMQ et Nginx montre comment construire un système performant, scalable et modulaire. Cette expérience illustre les principes fondamentaux des systèmes modernes utilisés dans les applications réelles.